

MODULE-8 SQL Assignment 1

1 .New Customers Acquired in June 2023

Business Problem: The marketing team ran a campaign in June 2023 and wants to see how many new customers signed up during that period.

```
SELECT p.PARTY_ID,  
       p.FIRST_NAME,  
       p.LAST_NAME,  
       cm.INFO_STRING,  
       tn.CONTACT_NUMBER,  
       p.CREATED_STAMP AS ENTRY_DATE  
  
FROM person p  
JOIN party_contact_mech pcm  
ON p.PARTY_ID = pcm.PARTY_ID  
JOIN contact_mech cm  
ON pcm.CONTACT_MECH_ID = cm.CONTACT_MECH_ID  
JOIN telecom_number tn  
ON cm.CONTACT_MECH_ID = tn.CONTACT_MECH_ID  
WHERE DATE(p.CREATED_STAMP) > '2023-05-31' AND DATE(p.CREATED_STAMP) <  
'2023-07-01';
```

The screenshot shows a SQL query execution interface. The query is as follows:

```
1 SELECT p.PARTY_ID,  
2       p.FIRST_NAME,  
3       p.LAST_NAME,  
4       cm.INFO_STRING,  
5       tn.CONTACT_NUMBER,  
6       p.CREATED_STAMP AS ENTRY_DATE  
7  
8 FROM person p  
9 JOIN party_contact_mech pcm  
10 ON p.PARTY_ID = pcm.PARTY_ID  
11 JOIN contact_mech cm  
12 ON pcm.CONTACT_MECH_ID = cm.CONTACT_MECH_ID  
13 JOIN telecom_number tn  
14 ON cm.CONTACT_MECH_ID = tn.CONTACT_MECH_ID  
15 WHERE DATE(p.CREATED_STAMP) > '2026-03-31' AND DATE(p.CREATED_STAMP) < '2026-05-01';  
16
```

The interface includes a "RUN" button, a "LIMIT: 1000" dropdown, a timer showing "00:00:00.29", and buttons for "SAVE", "COPY LINK", and a menu icon.

Below the query, there are tabs for "RESULTS" and "QUERY HISTORY". The "RESULTS" tab is active, showing a table with the following columns: PARTY_ID, FIRST_NAME, LAST_NAME, INFO_STRING, CONTACT_NUMBER, and ENTRY_DATE. The table contains one row of data:

PARTY_ID	FIRST_NAME	LAST_NAME	INFO_STRING	CONTACT_NUMBER	ENTRY_DATE
HOTWAX_USER	HotWax	User	NULL	724-0104	2026-04-28T04:57:29.781000

The interface also includes buttons for "CREATE CHART", "DOWNLOAD TO CSV", and "COPY TO CLIPBOARD", along with a "Filter results" input field.

2 List All Active Physical Products

Business Problem: Merchandising teams often need a list of all physical products to manage logistics, warehousing, and shipping.

```
SELECT p.PRODUCT_ID,  
       p.PRODUCT_TYPE_ID,  
       p.INTERNAL_NAME  
  
FROM product p  
JOIN product_type pt  
ON p.PRODUCT_TYPE_ID = pt.PRODUCT_TYPE_ID  
WHERE pt.IS_PHYSICAL = 'Y';
```

The screenshot shows a SQL query execution interface. At the top, the SQL query is entered in a text area. Below the query area, there are buttons for 'RUN', 'LIMIT: 1 000', a timer '00:00:00.24', 'SAVE', and 'COPY LINK'. Below these buttons, there are tabs for 'RESULTS' and 'QUERY HISTORY'. Under the 'RESULTS' tab, there are buttons for 'CREATE CHART', 'DOWNLOAD TO CSV', and 'COPY TO CLIPBOARD', along with a 'Filter results' input field. Below these buttons, the SQL query is repeated, followed by a '1k rows' indicator. At the bottom, there is a table with three columns: 'PRODUCT_ID', 'PRODUCT_TYPE_ID', and 'INTERNAL_NAME'. The table contains three rows of data.

PRODUCT_ID	PRODUCT_TYPE_ID	INTERNAL_NAME
M100068	FINISHED_GOOD	NULL
M100069	FINISHED_GOOD	NULL
M100070	FINISHED_GOOD	NULL

3 Products Missing NetSuite ID

Business Problem:

A product cannot sync to NetSuite unless it has a valid NetSuite ID. The OMS needs a list of all products that still need to be created or updated in NetSuite.

```
SELECT p.PRODUCT_ID,  
       p.PRODUCT_TYPE_ID,  
       p.INTERNAL_NAME,  
       gi.GOOD_IDENTIFICATION_TYPE_ID  
  
FROM product p  
JOIN good_identification gi  
ON p.PRODUCT_ID = gi.PRODUCT_ID  
WHERE gi.GOOD_IDENTIFICATION_TYPE_ID = 'ERP_ID' AND gi.ID_VALUE=NULL;
```

4 Product IDs Across Systems

Business Problem:

To sync an order or product across multiple systems (e.g., Shopify, HotWax, ERP/NetSuite), the OMS needs to know each system's unique identifier for that product. This query retrieves the Shopify ID, HotWax ID, and ERP ID (NetSuite ID) for all products.

```
SELECT p.PRODUCT_ID,  
       oi.EXTERNAL_ID AS SHOPIFY_ID,  
       oi.ORDER_ID AS HOTWAX_ID,
```

```
gi.ID_VALUE AS NETSUITE_ID
```

```
FROM order_item oi
```

```
JOIN product p
```

```
ON oi.PRODUCT_ID = p.PRODUCT_ID
```

```
JOIN good_identification gi
```

```
ON p.PRODUCT_ID = gi.PRODUCT_ID
```

```
WHERE gi.GOOD_IDENTIFICATION_TYPE_ID IN ('ERP_ID','NETSUITE_ID') ;
```

5 Completed Orders in August 2023

Business Problem:

After running similar reports for a previous month, you now need all completed orders in August 2023 for analysis.

```
SELECT oi.PRODUCT_ID,  
       p.PRODUCT_TYPE_ID,  
       oh.PRODUCT_STORE_ID,  
       oi.QUANTITY AS TOTAL_QUANTITY,  
       p.INTERNAL_NAME,  
       oisg.FACILITY_ID,  
       oh.EXTERNAL_ID,  
       f.FACILITY_TYPE_ID,  
       os.ORDER_STATUS_ID AS ORDER_HISTORY,  
       oh.ORDER_ID,  
       oi.ORDER_ITEM_SEQ_ID,  
       oisg.SHIP_GROUP_SEQ_ID
```

```
FROM order_header oh
```

```
JOIN order_item oi
```

```
ON oh.ORDER_ID = oi.ORDER_ID
```

```
JOIN order_item_ship_group oisg
```

```
ON oi.ORDER_ID = oisg.ORDER_ID AND oi.SHIP_GROUP_SEQ_ID =
```

```
oisg.SHIP_GROUP_SEQ_ID
```

```
JOIN product p
```

```
ON oi.PRODUCT_ID = p.PRODUCT_ID
```

```
LEFT JOIN facility f
```

```
ON oisg.FACILITY_ID = f.FACILITY_ID
```

```
LEFT JOIN order_status os
ON oh.ORDER_ID = os.ORDER_ID AND os.STATUS_ID = 'ORDER_COMPLETED'
WHERE os.STATUS_ID = 'ORDER_COMPLETED' AND oh.ORDER_DATE >= '2023-08-01
00:00:00'
AND oh.ORDER_DATE < '2023-09-01 00:00:00';
```

8 Payment Captured but Not Shipped

Business Problem:

Finance teams want to ensure revenue is recognized properly. If payment is captured but no shipment has occurred, it warrants further review.

```
SELECT oh.ORDER_ID,
       os.STATUS_ID,
       opp.STATUS_ID,
       ss.STATUS_ID AS "SHIPMENT STATUS"
FROM order_header oh
JOIN order_payment_preference opp
  ON oh.ORDER_ID = opp.ORDER_ID
JOIN order_status os
  ON opp.ORDER_PAYMENT_PREFERENCE_ID =
os.ORDER_PAYMENT_PREFERENCE_ID
JOIN order_shipment ors
  ON oh.ORDER_ID = ors.ORDER_ID
JOIN shipment_status ss
  ON ors.SHIPMENT_ID = ss.SHIPMENT_ID
WHERE os.STATUS_ID IN ('PAYMENT_RECEIVED','PAYMENT_SETTLED')
AND ss.STATUS_ID
IN('SHIPMENT_INPUT','SHIPMENT_PACKED','SHIPMENT_PICKED','SHIPMENT_SCHEDULED');
```

9 Orders Completed Hourly

Business Problem:

Operations teams may want to see how orders complete across the day to schedule staffing.

```
SELECT COUNT(DISTINCT oh.ORDER_ID) AS `TOTAL ORDERS`,  
       HOUR(os.STATUS_DATETIME) AS `HOUR`  
  
FROM ORDER_HEADER oh  
JOIN ORDER_STATUS os  
ON oh.ORDER_ID = os.ORDER_ID  
  
WHERE oh.STATUS_ID = 'ORDER_COMPLETED' AND os.STATUS_ID =  
'ORDER_COMPLETED'  
GROUP BY HOUR(os.STATUS_DATETIME)  
ORDER BY `HOUR` ASC;
```

The screenshot shows a SQL query editor with a query that counts distinct orders by hour. The query is as follows:

```
1 SELECT COUNT(DISTINCT oh.ORDER_ID) AS `TOTAL ORDERS`,  
2       HOUR(os.STATUS_DATETIME) AS `HOUR`  
3  
4 FROM ORDER_HEADER oh  
5 JOIN ORDER_STATUS os  
6 ON oh.ORDER_ID = os.ORDER_ID  
7  
8 WHERE oh.STATUS_ID = 'ORDER_COMPLETED' AND os.STATUS_ID = 'ORDER_COMPLETED'  
9 GROUP BY HOUR(os.STATUS_DATETIME)  
10 ORDER BY `HOUR` ASC;  
11
```

Below the query editor, there are buttons for **RUN**, **LIMIT: 1 000**, **00:00:00.44**, **SAVE**, **COPY LINK**, and a menu icon. The **RESULTS** tab is selected, showing the query history and a table of results. The table has two columns: **TOTAL ORDERS** and **HOUR**. The results are as follows:

TOTAL ORDERS	HOUR
2	0
3	1
1	2
4	3

The interface also includes a **CREATE CHART** button, a **DOWNLOAD TO CSV** button, a **COPY TO CLIPBOARD** button, and a **Filter results** input field. The query history shows the query being executed, and the results are displayed in a table with 11 rows.

10 BOPIS Orders Revenue (Last Year)

Business Problem:

BOPIS (Buy Online, Pickup In Store) is a key retail strategy. Finance wants to know the revenue from BOPIS orders for the previous year.

```
SELECT COUNT(oh.ORDER_ID) AS `TOTAL ORDERS`,
       SUM(oh.GRAND_TOTAL) AS `TOTAL REVENUE`

FROM ORDER_HEADER oh
WHERE oh.STATUS_ID = 'ORDER_COMPLETED'
AND YEAR(oh.ORDER_DATE) = YEAR(CURDATE()) - 1
AND EXISTS (
  SELECT 1
  FROM ORDER_ITEM_SHIP_GROUP oisg
  WHERE oisg.ORDER_ID = oh.ORDER_ID
  AND oisg.SHIPMENT_METHOD_TYPE_ID = 'STOREPICKUP'
);
```

11 Canceled Orders (Last Month)

Business Problem:

The merchandising team needs to know how many orders were canceled in the previous month and their reasons.

```
SELECT
  COUNT(DISTINCT oh.ORDER_ID) AS `TOTAL ORDERS`,
  os.CHANGE_REASON AS `CANCELATION REASON`
FROM ORDER_HEADER oh
JOIN ORDER_STATUS os
ON oh.ORDER_ID = os.ORDER_ID
WHERE oh.STATUS_ID = 'ORDER_CANCELLED'
  AND os.STATUS_ID = 'ORDER_CANCELLED'
  AND MONTH(os.STATUS_DATETIME) = MONTH(CURRENT_DATE - INTERVAL 1 MONTH)
GROUP BY os.CHANGE_REASON
ORDER BY `TOTAL ORDERS` DESC;
```

12 Product Threshold Value

Business Problem The retailer has set a threshold value for products that are sold online, in order to avoid over selling.

```
SELECT PRODUCT_ID,MINIMUM_STOCK AS `THRESHOLD`  
FROM PRODUCT_FACILITY  
WHERE MINIMUM_STOCK IS NOT NULL;
```

The screenshot shows a SQL query execution interface. At the top, the query is entered in a text area with line numbers 1 to 4. Below the query area, there are controls for running the query: a 'RUN' button, a 'LIMIT' dropdown set to '1 000', a green timer showing '00:00:00.37', a 'SAVE' button, a dropdown arrow, a 'COPY LINK' button, and a three-dot menu. Below these controls, there are two tabs: 'RESULTS' (selected) and 'QUERY HISTORY'. Under the 'RESULTS' tab, there are three buttons: 'CREATE CHART', 'DOWNLOAD TO CSV', and 'COPY TO CLIPBOARD', followed by a 'Filter results' input field. Below the buttons, the query is repeated: 'SELECT PRODUCT_ID, MINIMUM_STOCK AS `THRESHOLD` FROM PRODUCT_FACILITY WHERE MINIMUM_STOCK IS NOT NU...'. To the right of the query is a button labeled '3 rows'. Below the query, a table displays the results with two columns: 'PRODUCT_ID' and 'THRESHOLD'. The table contains three rows of data.

PRODUCT_ID	THRESHOLD
M103007	10000
M103167	10000
M103647	10000

7 Newly Created Sales Orders and Payment Methods

Business Problem:

Finance teams need to see new orders and their payment methods for reconciliation and fraud checks.

```
SELECT oh.ORDER_ID,  
       opp.MAX_AMOUNT,  
       opp.PAYMENT_METHOD_TYPE_ID,  
       oh.EXTERNAL_ID AS "SHOPIFY ORDER ID"  
FROM order_header oh  
JOIN order_payment_preference opp  
ON oh.ORDER_ID = opp.ORDER_ID;
```

1

2

3

4

5

6

7

SELECT

oh.ORDER_ID,

opp.MAX_AMOUNT,

opp.PAYMENT_METHOD_TYPE_ID,

oh.EXTERNAL_ID AS "SHOPIFY ORDER ID"

FROM order_header oh

JOIN order_payment_preference opp

ON oh.ORDER_ID = opp.ORDER_ID;

RUN

LIMIT: 1 000

00:00:00.31

SAVE

COPY LINK

RESULTS QUERY HISTORY

CREATE CHART

DOWNLOAD TO CSV

COPY TO CLIPBOARD

Filter results

SELECT oh.ORDER_ID, opp.MAX_AMOUNT, opp.PAYMENT_METHOD_TYPE_ID, oh.EXTERNAL_ID AS "SHOPIFY ORDER ID...

139 rows

ORDER_ID	MAX_AMOUNT	PAYMENT_METHOD_TYPE_ID	SHOPIFY ORDER ID
1	699.95	EXT_SHOP_OTHR_GTWAY	7119825371453
2	785.95	EXT_SHOP_OTHR_GTWAY	7119886745917
3	1073	EXT_SHOP_OTHR_GTWAY	7127896097085
4	99	EXT_SHOP_OTHR_GTWAY	7123389219133